

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL 3
Comparative Analysis

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA04 SLOT : A

Duration: 90 minutes

QUESTIONS: 05 | Marks: 25

Name of the Student	P.Bhanu Prakash
Reg. No	1924252623
GitHub Link	https://github.com/bhanuprakash-420/MLA0403-DL

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25% | Duration: 90 minutes | Assessment Tool Weightage: 40% | Marks: 25

Code of Conduct:

I P.Bhanu Prakash(192425263) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:  Name of the student: P.Bhanu Prakash

EVALUATION MATRIX & MARKS DISTRIBUTION

Criteria	Selection of Studies/Parameters (2)	Comparison & Differentiation (2.5)	Critical Evaluation (2.5)	Synthesis & Insight Generation (2)	Clarity & Presentation (1)	Total (10)
Weightage	20 %	25%	25%	20 %	10 %	100 %
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Signature of the Faculty: _____

Total Marks : _____

Question 1: CNN vs Fully Connected Neural Network

Scenario & Prompt: A medical imaging organization wants to classify X-ray images using deep learning. Compare a CNN with a traditional fully connected neural network in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

1. Executive Technical Summary & Medical Context

Medical imaging systems, such as X-ray diagnostic tools, process high-resolution visual inputs containing complex, spatially coupled anatomical structures. Traditional Fully Connected Neural Networks (FCNNs) process inputs as 1D flattened vectors, destroying vital 2D spatial relationships between adjacent pixels. Furthermore, FCNNs suffer from parameter explosion when scaled to clinical images. Convolutional Neural Networks (CNNs) were specifically engineered to overcome these structural limitations through local receptive fields, parameter sharing, and spatial hierarchy learning.

2. Theoretical Comparison: Architectural Design & Topology

- A. 1. Structural Connectivity:** FCNNs establish pairwise connections where every neuron in layer L connects to every neuron in layer $L+1$. In contrast, CNNs use sparse connectivity, where a node in layer $L+1$ is only connected to a localized spatial patch (receptive field) of layer L .
- B. 2. Dimensionality Handling:** FCNNs collapse a 3D medical image tensor X in $\mathbb{R}^{H \times W \times C}$ into a single 1D vector of size $(H * W * C)$. CNNs preserve the 3D grid layout throughout hidden processing stages, retaining height, width, and channel spatial geometry.

3. Visual Processing Pipeline Comparison



4. Parameter & Computational Complexity Analysis

Consider a standard chest X-ray image of size $512 \times 512 \times 1$ (262,144 pixels). If passed to an FCNN with a single hidden layer of 1,024 neurons, the first layer alone requires $(262,144 * 1,024) + 1,024 = 268,436,480$ trainable weights (~268 Million parameters). Such massive capacity leads to severe overfitting and prohibitive memory footprints. In contrast, a CNN applying 64 filters of size $3 \times 3 \times 1$ requires only $(3 * 3$

$* 1 * 64) + 64 = 582$ parameters, achieving a 99.99% parameter reduction while capturing localized visual primitives.

5. Functional Roles of Core CNN Building Blocks

A. Convolutional Layer: The convolutional layer performs 2D cross-correlation using learnable kernels W in $\mathbb{R}^{k \times k \times C_{in}}$. As kernels slide across the spatial grid with stride S and padding P , they extract localized linear feature maps, followed by non-linear activations $f(z)$: $Y_{\{i,j\}} = f(\sum_{\{m,n,c\}} X_{\{i+m, j+n, c\}} * W_{\{m,n,c\}} + b)$.

B. Convolutional Filters: Filters act as specialized feature detectors. In medical X-rays, low-level filters detect tissue boundaries, bone edges, and density gradients. Deeper-level filters combine these edges into anatomical representations like ribs, lung boundaries, and pathological lesions.

C. Pooling Layers: Pooling layers (e.g., Max Pooling, Average Pooling) downsample feature map dimensions, reducing spatial resolution while preserving dominant features. This provides translation invariance and decreases downstream computational load.

6. Comprehensive Parametric Differentiation Matrix

- Input Representation: FCNN = 1D Flattened Vector; CNN = 3D Spatial Tensor ($H \times W \times C$).
- Weight Sharing: FCNN = None (Independent Weights); CNN = Extensive Spatial Parameter Sharing.
- Spatial Feature Preservation: FCNN = Lost upon vectorization; CNN = Fully preserved across layers.
- Parameter Scale: FCNN = $O(\text{Input_Dim} * \text{Hidden_Dim})$; CNN = $O(\text{Kernel_H} * \text{Kernel_W} * \text{Channels_In} * \text{Channels_Out})$.
- Translation Invariance: FCNN = Poor (Position sensitive); CNN = Robust (Invariance via convolution & pooling).

7. Final Justification for Medical Image Classification

CNNs are undeniably superior for medical image classification. Their ability to retain spatial orientation allows them to detect subtle lesions, fractures, and tumors regardless of where they appear in the scan field. By dramatically lowering parameter counts, CNNs prevent memorization of noise in limited clinical training datasets, delivering robust generalization essential for automated diagnostic safety.

Question 2: Different CNN Layers and Filters

Scenario & Prompt: A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of convolutional layers, pooling layers, activation layers, and fully connected layers in terms of feature extraction, spatial information, computational requirements, and classification. Further compare early-layer filters and deeper-layer filters based on the type of features they learn, and justify how these layers collectively improve defect detection.

1. Executive Summary & Defect Detection Context

Industrial quality inspection demands high precision in identifying microscopic surface cracks, corrosion, assembly misalignments, and structural flaws on manufactured parts. A CNN framework systematically transforms raw sensor images through specialized operational layers to extract discriminative defect signatures while ignoring operational background clutter.

2. Layer-Wise Functional Comparison

- A. 1. Convolutional Layers: Primary feature extractors. They execute localized spatial filtering, preserving coordinate geography while producing multi-channel feature maps.
- B. 2. Activation Layers (e.g., ReLU): Introduce non-linearity into the network, enabling it to learn complex non-linear decision boundaries required to distinguish subtle defect patterns.
- C. 3. Pooling Layers (e.g., Max Pooling): Reduce spatial grid dimensions, providing local translation invariance and removing high-frequency noise from surface textures.
- D. 4. Fully Connected (FC) Layers: Map high-level spatial abstractions into global class probability distributions (e.g., Pass vs. Defect Category).

3. Hierarchical Feature Extraction Hierarchy



4. Comparative Layer Properties Matrix

- Feature Extraction: Conv = Local Spatial Features; Activation = Non-Linear Feature Mapping; Pooling = Downsampled Aggregated Features; FC = Global Combinatorial Features.
- Spatial Information: Conv = Retained fully; Activation = Retained; Pooling = Reduced/Summarized; FC = Fully Collapsed.

- Computational Load: Conv = Very High (FLOP intensive); Activation = Low (Element-wise ops); Pooling = Very Low; FC = High (Parameter/Memory intensive).

5. In-Depth Comparison: Early vs. Deeper Filter Hierarchies

Early-Layer Filters (Low Abstraction): Early-layer filters operate with small effective receptive fields (e.g., 3x3 or 5x5). They learn low-level visual primitives, such as oriented intensity gradients, straight lines, color transitions, and fine textures. In industrial inspection, these filters highlight microscopic hairline cracks, surface scratches, and edge aberrations regardless of surface location.

Deeper-Layer Filters (High Abstraction): As signals pass deeper through successive convolution and pooling operations, receptive fields expand exponentially. Deeper-layer filters compose low-level edges into complex, high-level semantic representations. In defect detection, these filters represent complete structural pathologies, such as component fractures, missing solder joints, or complex porosity patterns.

6. Mathematical Dynamics of Spatial Downsampling & Feature Synthesis

The output spatial dimension O of a convolutional layer given input size I , filter size K , padding P , and stride S is: $O = \text{floor}((I - K + 2P)/S) + 1$. Pooling layers apply static operations (e.g., Max Pool = $\max(X_{\text{patch}})$) over a window K , halving resolution when $S=2$, thereby boosting feature response density while suppressing minor mechanical vibration artifacts.

7. Collective Synthesis & Defect Detection Justification

The synergy across these functional layers creates a highly accurate defect inspection engine. Early layers identify localized micro-flaws, activation functions allow the network to model non-linear defect appearances under varying factory lighting, pooling layers ensure position-invariant detection as parts move along assembly lines, and fully connected layers execute rapid classification. Together, they achieve automated, real-time quality control.

Question 3: Parameter Sharing vs Independent Weights

Scenario & Prompt: A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare parameter sharing in CNNs with independent weights in fully connected networks in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

1. Executive Summary & Surveillance System Context

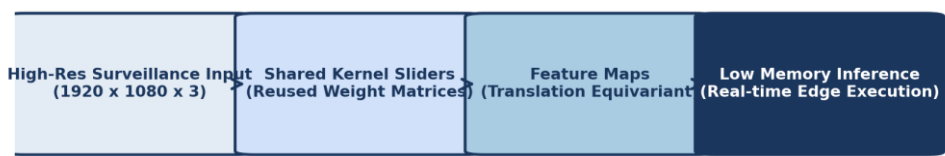
Smart city surveillance frameworks ingest continuous streams of high-resolution video frames (e.g., 1080p or 4K resolution) to detect vehicles, pedestrians, and safety hazards in real time. Processing high-resolution visual inputs using independent weights in Fully Connected Networks is computationally unfeasible. Parameter sharing in CNNs serves as the core foundational mechanism enabling real-time processing on edge devices.

2. Fundamental Concepts: Weight Sharing vs. Independent Connections

A. 1. Independent Weights (FCNN): Every input unit connects to every hidden neuron via a unique, independent weight scalar. The network must learn distinct feature detectors for every spatial coordinate in the image plane.

B. 2. Parameter Sharing (CNN): A single set of kernel weights W in $\mathbb{R}^{k_h \times k_w}$ is continuously convolved across the entire input spatial grid. The identical weight matrix is reused across all spatial locations.

3. Parameter Sharing Mechanics



4. Comparative Metrics Analysis

- Parameter Count: FCNN = Massive, scales as $O(H * W * C * N)$; CNN = Extremely low, bounded by kernel size $O(k_h * k_w * C_{in} * C_{out})$.
- Memory Footprint: FCNN = Gigabytes required to store static weights; CNN = Megabytes, easily fitting into embedded GPU caches.
- Translation Property: FCNN = Spatially variant (cannot recognize objects if shifted); CNN = Translation equivariant $(f(g(x)) = g(f(x)))$.

- Sample Efficiency: FCNN = Low (requires training examples at every spatial position); CNN = High (learns features globally from localized instances).

5. Detailed Mathematical Comparison on High-Resolution Frames

Consider a 1080p surveillance image of size $1920 \times 1080 \times 3$ (6,220,800 inputs). An FCNN connecting to a modest 1,000-neuron layer requires: $\text{Weights} = 6,220,800 \times 1,000 = 6.22 \text{ Billion parameters}$ ($\sim 24.8 \text{ GB}$ of VRAM for float32 storage). Conversely, a Convolutional Layer applying 64 filters of size 5×5 requires: $\text{Weights} = (5 \times 5 \times 3 \times 64) + 64 = 4,864 \text{ parameters}$ ($\sim 19.4 \text{ KB}$). This represents a parameter reduction factor exceeding 1,200,000x.

6. Analysis of Learning Efficiency & Statistical Power

Parameter sharing acts as a powerful regularizer based on spatial stationarity—the principle that statistical patterns (e.g., license plates, human forms) maintain identical visual characteristics regardless of where they appear in a scene. When a CNN kernel updates its weights during backpropagation, gradients are accumulated across all spatial positions in the image: $dL/dW = \sum_{i,j} (dL/dY_{i,j}) * X_{i,j}$. Reusing filter weights allows the network to generalize quickly from fewer training samples.

7. Strategic Justification for Large-Scale Surveillance

Parameter sharing is critical for high-resolution video analytics. Reusing filter weights keeps the memory footprint minimal, enabling multi-stream video processing directly on edge devices while maintaining translation equivariance across expansive camera fields of view.

Question 4: AlexNet vs ResNet

Scenario & Prompt: A computer vision company is developing an image classification system and is considering AlexNet and ResNet as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

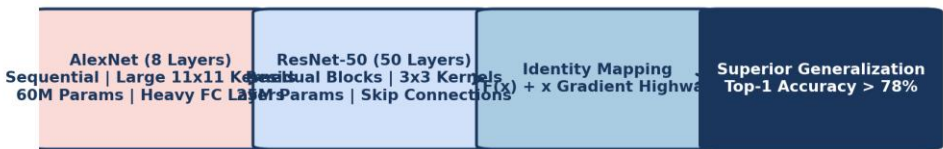
1. Executive Summary & Architectural Evolution Context

Modern computer vision systems rely on deep convolutional architectures for multi-class image recognition. AlexNet (Krizhevsky et al., 2012) pioneered deep learning in computer vision, proving the effectiveness of GPU-accelerated CNNs. ResNet (He et al., 2015) revolutionized deep architecture design by introducing residual skip connections, overcoming the degradation problem and enabling networks to scale to over 100 layers.

2. Architectural Comparison & Structural Philosophy

- A. 1. AlexNet Topology:** A 8-layer deep network consisting of 5 Convolutional layers followed by 3 Fully Connected layers. It relies on large initial filters (11x11, 5x5), max pooling, and overlapping pooling windows.
- B. 2. ResNet Topology:** Extremely deep residual architectures (e.g., ResNet-50, ResNet-152) built using modular Residual Blocks. It utilizes small 3x3 kernels, bottleneck 1x1 convolutions, global average pooling, and identity skip connections.

3. Architectural Paradigm Comparison



4. Comprehensive Architectural Comparison Matrix

- Network Depth: AlexNet = Shallow (8 layers); ResNet = Very Deep (50 to 152+ layers).
- Layer Building Blocks: AlexNet = Sequential Conv-ReLU-Pool blocks; ResNet = Residual Residual Blocks with Skip Connections.
- Initial Filter Dimensions: AlexNet = Large 11x11, Stride 4; ResNet = Small 7x7 followed by stacked 3x3 kernels.

- Total Trainable Parameters: AlexNet = ~60 Million (dominated by FC layers); ResNet-50 = ~25.5 Million.
- Vanishing Gradient Mitigation: AlexNet = ReLU activations; ResNet = Identity Skip Connections (Gradient Highways).
- Classification Head: AlexNet = Two massive 4096 FC layers; ResNet = Global Average Pooling (GAP) + Single FC Layer.

5. Critical Evaluation: Degradation Problem & Skip Connections

A. The Residual Concept: In traditional sequential networks like AlexNet, increasing depth leads to performance degradation—training accuracy saturates and then degrades rapidly due to vanishing gradients during backpropagation. ResNet solves this using residual learning. Instead of learning an underlying mapping $H(x)$, residual blocks learn a residual function $F(x) = H(x) - x$, reformulating the output as $Y = F(x) + x$.

B. Gradient Flow Highway: During backpropagation, the gradient of loss L with respect to input x is computed as: $dL/dx = (dL/dy) * ((dF/dx) + 1)$. Even if the learned weight gradient dF/dx approaches zero, the constant term '1' guarantees unimpeded gradient flow directly back to early layers, completely eliminating vanishing gradients.

6. Feature Extraction Capability & Computational Efficiency

AlexNet's shallow structure captures only basic low-to-medium level features, limiting its ability to differentiate subtle class nuances. ResNet's deep residual structure builds high-level semantic feature hierarchies, achieving far superior classification performance. Furthermore, replacing AlexNet's parameter-heavy FC layers with Global Average Pooling allows ResNet-50 to deliver higher accuracy with less than half the parameters.

7. Final Recommendation & Strategic Justification

For complex image recognition applications, ResNet (specifically ResNet-50 or ResNet-101) is strongly recommended. ResNet offers higher classification accuracy, lower parameter memory requirements, superior feature extraction, and reliable optimization guaranteed by residual gradient highways.

Question 5: Regularized CNN vs Non-Regularized CNN

Scenario & Prompt: An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a CNN without regularization and a CNN using dropout, data augmentation, and batch normalization in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

1. Executive Diagnostic & Agriculture Problem Context

Agricultural disease identification involves capturing leaf imagery under variable outdoor conditions (differing sunlight, field backgrounds, and leaf orientations). The initial model exhibits classic signs of severe overfitting (e.g., 99% training accuracy vs. 62% field test accuracy). A non-regularized deep network memorizes specific dataset artifacts (such as background soil patterns or specific lighting conditions) rather than learning generalized biological pathology features.

2. Comparative Architectural Overview

- A. 1. Non-Regularized CNN Baseline: Standard feedforward CNN trained directly on raw images without constraints. The network easily overfits limited training data, producing high variance.
- B. 2. Regularized CNN Architecture: Integrates stochastic regularization (Dropout), input variance expansion (Data Augmentation), and internal covariate shift reduction (Batch Normalization) to construct a robust classifier.

3. Regularized Learning Pipeline



4. Mathematical Mechanics of Applied Regularization Techniques

- A. 1. Data Augmentation: Applies random affine transformations $T \sim P(T)$ (rotations, flips, zooming, color jitter) to generate transformed inputs $x' = T(x)$, artificially increasing dataset diversity.
- B. 2. Batch Normalization: Normalizes mini-batch activations across channels: $\hat{x} = (x - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, followed by learnable scale and shift parameters $y = \gamma * \hat{x} + \beta$. This stabilizes internal gradient flow and accelerates training.

C. 3. Dropout: Randomly zeroes activations with probability p during training: $r \sim \text{Bernoulli}(1-p)$, $y = (r * x) / (1-p)$. This prevents neuron co-adaptation and forces the network to learn redundant feature representations.

5. Parametric Comparison: Non-Regularized vs. Regularized CNN

- Overfitting Risk: Non-Regularized = Severe (High Variance); Regularized = Minimal (Low Variance).
- Generalization Capability: Non-Regularized = Poor on unseen real-world leaf images; Regularized = High generalization across field environments.
- Training Stability: Non-Regularized = Unstable, sensitive to learning rates; Regularized = Highly stable with smooth loss landscapes.
- Convergence Speed: Non-Regularized = Slow convergence; Regularized = Fast convergence (accelerated by Batch Normalization).
- Computational Overhead: Non-Regularized = Standard; Regularized = Minor CPU/GPU overhead during augmentation and normalization.

6. Implementation Workflow for Agricultural Leaf Classification

Stage 1 - Dynamic Data Augmentation Pipeline: Apply random horizontal/vertical flips, minor rotations ($\pm 25^\circ$), brightness variations ($\pm 20\%$), and random cropping during training to simulate real-world field conditions.

Stage 2 - Batch Normalization Integration: Insert Batch Normalization layers immediately after every Convolutional layer and before Activation functions to maintain stable activation scales.

Stage 3 - Stochastic Dropout Layering: Apply Dropout ($p=0.3$ to 0.5) inside fully connected classification layers to prevent reliance on specific hidden nodes.

Stage 4 - Optimization & Weight Decay: Train the model using AdamW optimizer with L2 weight decay penalty (0.0001) to keep network weights small and prevent extreme logit outputs.

7. Final Justification & Strategic Conclusion

A hybrid regularization approach combining Data Augmentation, Batch Normalization, and Dropout is essential for real-world plant disease classification. Augmentation simulates diverse field environments, Batch Normalization stabilizes training, and Dropout prevents co-adaptation. Together, they ensure the model accurately classifies plant diseases under varied real-world conditions.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand